

INSTILLER: Toward Efficient and Realistic RTL Fuzzing

Gen Zhang¹, Pengfei Wang¹, Tai Yue¹, Danjun Liu, Yubei Guo, and Kai Lu

Abstract—Bugs exist in hardware, such as CPU. Unlike software bugs, these hardware bugs need to be detected before deployment. Previous fuzzing work in CPU bug detection has several disadvantages, e.g., the length of register transition level (RTL) input instructions keeps growing, and longer inputs are ineffective for fuzzing. In this article, we propose INSTILLER (Instruction Distiller), an RTL fuzzer based on ant colony optimization (ACO). First, to keep the input instruction length short and efficient in fuzzing, it distills input instructions with a variant of ACO (VACO). Next, related work cannot simulate realistic interruptions well in fuzzing, and INSTILLER solves the problem of inserting interruptions and exceptions in generating the inputs. Third, to further improve the fuzzing performance of INSTILLER, we propose hardware-based seed selection and mutation strategies. We implement a prototype and conduct extensive experiments against state-of-the-art fuzzing work in real-world target CPU cores. In experiments, INSTILLER has 29.4% more coverage than DiFuzzRTL. In addition, 17.0% more mismatches are detected by INSTILLER. With the VACO algorithm, INSTILLER generates 79.3% shorter input instructions than DiFuzzRTL, demonstrating its effectiveness in distilling the input instructions. In addition, the distillation leads to a 6.7% increase in execution speed on average.

Index Terms—Fuzzing, hardware security, register transition level (RTL).

I. INTRODUCTION

CPU BUGS are notorious. Besides the well-known Meltdown and Spectre [1], numerous bugs are reported, including the Pentium FDIV bug [2], Broadwell MCE bug [3], and Ryzen segfault bug [4]. All of them can cost the manufacturers millions or billions of dollars in mitigating and repairing the bugs.

Before CPU deployment, the circuits and register transition level (RTL) designs should be thoroughly verified. In software deployment, bugs can be avoided with timely patches. However, in the development of CPU, once deployed, it is nearly impracticable to remove the impact of hardware vulnerabilities. For example, the mitigation of Meltdown and Spectre only focuses on part of the mainstream products,

due to the challenging balance among the mitigation itself, performance impact, and implementation complexity [5].

Previous work has made some attempts to detect CPU bugs [6], [7], both in static and dynamic techniques. Among them, verifying CPU with fuzz testing [8] is one of the most promising approaches [9], [10]. However, there are still several drawbacks to these techniques, and they extend to the following challenges.

Challenge 1 (Growing Input Length): The basic structure of the input instructions of the CPU consists of an instruction sequence. Interruptions and exceptions can be inserted to simulate the real-world execution of the CPU. Starting from a simple instruction, as the fuzzing process goes on, the length of input tends to increase. The speed of fuzz testing is the key to its success [11], [12], [13], [14], [15], [16]. Longer input instructions are catastrophic to the execution speed of fuzzing since longer inputs will spend more CPU cycles. More importantly, according to our analysis in the evaluation in Section V-C, coverage does not increase proportionally to input length. Therefore, we should come up with solutions to shorten the input instructions and improve the fuzzing efficiency.

Challenge 2 (Realistic Interruption and Exception Handling): Interruptions and exceptions are common in the execution of the CPU. Simulating them in testing CPU can cover the corner cases of CPU verification. Previous fuzzing work mentioned considering interruptions in the design [10], but the approach is relatively simple. Exceptions are not simulated, and multiple interruptions and exceptions and their priorities are also not included. Missing these situations cannot simulate the real-world CPU execution and cannot cover all the CPU states.

Challenge 3 (Fuzzing Techniques Related to Hardware): Fuzzing techniques are initially designed for testing software. Many customized approaches in fuzzing, such as program transformation [17] and process tracing [18], are also tailored for software programs. Especially in the critical steps of fuzzing, such as seed selection and mutation, previous fuzzing work did not combine fuzzing with hardware features well. For example, Hur et al. [10] did not consider seed selection when fuzzing the CPU. Not using hardware-related techniques cannot improve the fuzzing performance in testing CPU RTL.

To address the above challenges, we propose the following techniques.

Input Instruction Distillation Based on a Variant of Ant Colony Optimization (ACO): To save the CPU cycles and improve fuzzing performance, we need to distill the inputs and shorten the input instruction length. The basic idea of input instruction distillation is to construct a subset of the original input set, which is shorter in length and can maintain the original coverage. ACO is one of the latest techniques for approximate optimization [19]. The algorithm simulates the

Manuscript received 21 August 2023; revised 21 December 2023 and 23 January 2024; accepted 26 January 2024. Date of publication 31 January 2024; date of current version 20 June 2024. This work was supported in part by the Natural Science Foundation of China under Grant 61902412 and Grant 61902416; in part by the Research Project of National University of Defense Technology under Grant ZK20-17, Grant ZK20-09, and Grant ZK23-14; and in part by the Natural Science Foundation of Hunan Province of China under Grant 2021JJ40692. This article was recommended by Associate Editor G. Di Natale. (Corresponding authors: Pengfei Wang; Kai Lu.)

The authors are with the College of Computer, National University of Defense Technology, Changsha 410073, China (e-mail: zhanggen@nudt.edu.cn; pfwang@nudt.edu.cn; kailu@nudt.edu.cn).

Digital Object Identifier 10.1109/TCAD.2024.3360318

routine of an ant colony to search for the shortest path to the target city. We use the idea of ACO to distill input instructions. We model the length of input instructions as the number of ants and the RTL circuits as cities. The algorithm can output the best input instruction and length for the current status, which finishes the task of input instruction distillation. Moreover, we make some changes to the classic ACO and propose a variant of ACO (VACO) to fit the RTL fuzzing scenario.

Simulating Realistic Interruption and Exception Handling: First, we include exceptions in fuzzing the CPU, which is not proposed in previous work [10]. Next, we integrate more than one interruption and exception to test the CPU, aiming to simulate the real-world execution scenario of the CPU more comprehensively. Besides, we consider the priorities of different interruptions and exceptions, which can thoroughly fuzz the CPU. The above techniques can better simulate real-world interruption and exception handling than previous work.

Hardware-Related Seed Selection and Mutation: We propose new seed selection and mutation strategies in fuzzing the CPU. In seed selection, not only basic fuzzing heuristics are taken into consideration but also hardware heuristics, e.g., special instructions and registers. For mutation, we propose strategies closely related to hardware, such as insertion or deletion based on the input instruction length. The seed selection and mutation strategies combine fuzzing with hardware characteristics, and they overcome the drawbacks of previous tools.

We implement a prototype INSTILLER (Instruction Distiller) and conduct extensive evaluation against state-of-the-art fuzzing work. In general, the results show the effectiveness of our proposed techniques. Our tool increases coverage by 29.4%. For input instruction distillation, the length of INSTILLER is 79.3% shorter than DiFuzzRTL. For vulnerability discovery, INSTILLER finds 17.0% more mismatches in the targets. In addition, the input instruction distillation leads to a 6.7% increase in execution speed on average.

In conclusion, we make the following contributions to this article.

- 1) We propose an input instruction distillation technique, which is based on a VACO. The distillation can make the inputs shorter and more effective.
- 2) We enable our fuzzer to handle multiple interruptions and exceptions. The priorities of them are also considered. These techniques can simulate realistic interruption and exception handling well.
- 3) We propose hardware-based seed selection and mutation strategies. We use hardware-related heuristics and mutation operations to improve fuzzing performance in the situation of RTL fuzzing.
- 4) We implement a prototype named INSTILLER and conduct extensive experiments. The results show that our tool outperforms previous work and demonstrate the effectiveness of our proposed approaches.

II. BACKGROUND AND MOTIVATION

A. CPU Design, Interruptions, and Exceptions

CPU Design and Verification: Instruction set architecture (ISA) is the basis of designing a CPU. Different types of ISA can be implemented on different processors, e.g., Intel (X86) and M1 (ARM). RTL is a real hardware design based on the specific ISA. RTL can be described in hardware description

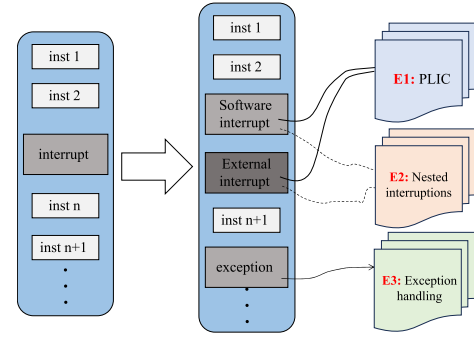


Fig. 1. Example of the effectiveness of multi-interruption and exception hardware fuzzing.

languages (HDLs) such as Verilog. Most importantly, CPU RTL should be completely tested before deployment. Not like software, the bugs and vulnerabilities in hardware cannot be easily mitigated with patches. Dynamic techniques are more common in testing RTL [10], which include testing with ISA and RTL simulation. The idea of differential testing in INSTILLER is to compare the results of ISA and RTL simulation and detect hardware bugs.

Input Instructions, Interruptions, and Exceptions: Before execution, specific instructions should be loaded into the CPU. In this article, the inputs of CPU RTL are formed with different instructions. By interpreting every instruction, the CPU finishes executing the program. Besides normal executions, interruptions and exceptions will raise, e.g., IO interruptions and illegal access exceptions. When testing RTL, interruptions and exceptions should be simulated to thoroughly cover the corner situations. Moreover, multiple interruptions and exceptions and their priorities are common in CPU execution. For example, another high-priority interruption occurs when handling a low-priority one. Previous fuzzing work [9], [10] failed to handle multiple interruptions or exceptions and their priorities. We consider these situations in this article.

Fig. 1 is a motivating example showing how a single-interruption no-exception strategy limits the effectiveness of fuzzing. This is a simplified input sequence containing instructions, two interruptions with different privilege levels, and an exception. There are mainly three aspects where the single-interruption no-exception strategy limits the effectiveness of fuzzing, i.e., coverage. First, platform level interrupt controller (PLIC) is a subsystem that can control the arbitration and distribution of multiple interrupts [20]. The code in this subsystem can only be triggered when there are multiple interruptions in the input sequence. Thus, single-interruption fuzzers fail to reach this coverage, which is denoted as *E1* in Fig. 1.

Moreover, the single-interruption no-exception strategy also hinders the fuzzers from covering nested interruption scenarios, which is denoted as *E2*. Handling nested interruptions is enabled in certain IP cores [21]. Single interruption can never reach these scenarios. Therefore, enabling multiple and nested interruptions in CPU fuzzing can reach more coverage.

Third, exception handling is an important subsystem in CPU implementation [22], and there are at least 16 entries for standard exception handling. No-exception fuzzing strategy fails to cover this exception-handling code. Our proposed fuzzing strategy with exceptions in the input sequence can solve this problem, which is denoted as *E3* in Fig. 1.

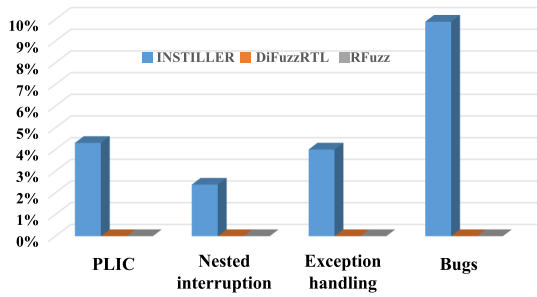


Fig. 2. Percentages of HDL line coverage and bugs of INSTILLER, DiFuzzRTL, and RFuzz related to multiple interruptions and exceptions.

Moreover, a preliminary experiment is conducted to reveal the insight of multiple interruptions and exceptions. We compare INSTILLER with state-of-the-art DiFuzzRTL and RFuzz to collect the coverage increase and bug detection related to interruptions and exceptions in Fig. 2. DiFuzzRTL and RFuzz are not designed with multiple interruptions and exceptions. Therefore, their related line coverage and detected bugs are zeros. The percentages of HDL lines covered by INSTILLER are 4.3%, 2.4%, and 4.0% of the overall coverage, in PLIC, nested interruptions, and exception handling, respectively. In total, HDL line coverage related to multiple interruptions and exceptions makes up for over 10% of the coverage by INSTILLER. Besides, bugs triggered by multiple interruptions and exceptions are about 10% of all the bugs. The results in Fig. 2 are *quantitative examples* of the motivation in Fig. 1.

Therefore, we believe enabling multiple interruptions and exceptions with their priorities is innovative compared with other fuzzers in two aspects. First, the fuzzing strategy in [9] and [10] is immature without multiple interruptions and exceptions with their priorities, and they deviate significantly from the actual CPU execution scenario. This is the key difference between our proposed “REALISTIC” strategy and theirs. Second, we have invested sufficient effort in code implementation in this part. For instance, the insertion of multiple interruptions with privilege levels requires us to deal with the *mcause* register and other issues. Therefore, our strategy is creative compared with previous work.

B. Fuzzing and Differential Testing

Fuzzing or fuzz testing [8] is one of the most successful software testing techniques. Coverage-guided grey-box fuzzing (CGF) is a variant of fuzzing, and it is famous for its great balance between effectiveness and efficiency [23]. The basic steps of fuzzing include the following.

- 1) Given initial seeds.
- 2) Select a seed and mutate the seed to generate input instructions.
- 3) Execute the target program with the instructions.
- 4) If there is new coverage, save the input instruction to the seed pool; If there is a program crash, report and save this crash.
- 5) Continuing to step 2).

Differential testing compares the results of two or more systems, and different results indicate bugs and vulnerabilities [24]. Based on it, differential fuzz testing is proposed to solve more complex bug-discovery problems [10].

Algorithm 1 Overview of the Procedures of INSTILLER

Require: Initial seeds S

```

1: while  $t < TIME\_OUT$  do
2:   if  $start\_distill == True$  then
3:      $re = relation\_extract()$ 
4:      $len = VACO(re)$ 
5:   end if
6:    $s = seed\_selection(seed)$ 
7:    $s' = mutation(s, len)$ 
8:    $input = interrupt\_exception(s')$ 
9:    $O_I = ISA\_sim(input)$ 
10:   $O_R = RTL\_sim(input)$ 
11:   $Cross\_check(O_I, O_R)$ 
12: end while

```

Ensure: Bug reports

In CPU verification, comparing the results of RTL executions with that of a golden model (ISA) is an effective testing technique [10], [25]. Therefore, our work combines the idea of CGF and differential testing to boost the CPU verification process.

C. Ant Colony Optimization

ACO was first introduced in the early 1990's [19]. It is one of the latest techniques for approximate optimization. ACO is used to solve combinatorial optimization problems, such as the traveling salesman problem (TSP), which is an optimization algorithm simulating ant foraging behavior.

The basic procedure of ACO can be concluded as follows.

- 1) Initialize the parameters.
- 2) Calculate the probability for every city and every ant.
- 3) Select the best next city to walk for every ant.
- 4) Update the pheromone table after the ants finish walking.
- 5) Continue to step 2) before termination.

In this article, we use the idea of ACO to distill input instructions. However, classic ACO does not fit RTL testing. The number of ants in ACO is constant, and it is variable in our model. Moreover, in classic ACO, the algorithm chooses the next best city for an ant. In our work, executing an input instruction will cover multiple circuits (cities), and it is choosing the next best cities. Therefore, we propose a variant of classic ACO to handle the above problems.

III. DESIGN

A. Overview

Fig. 3 is the overview of INSTILLER. There are mainly three newly designed infrastructures, including the VACO algorithm, interruption and exception simulation, and seed selection with mutation. First, VACO is capable of distilling the input instructions in RTL fuzzing, which can keep the input short and effective. Next, realistic interruption and exception handling are simulated by our simulation process. Through this kind of simulation, our fuzzing process is closer to the real-world execution of the CPU. In addition, the seed selection and mutation strategies integrate hardware-related features into fuzzing and improve the fuzzing performance.

The detailed execution process of INSTILLER is shown in Algorithm 1. Given initial seeds, the fuzzing process is started. Depending on the current coverage status, INSTILLER decides whether the input instruction distillation should be started. Distillation includes relationship extraction and the VACO

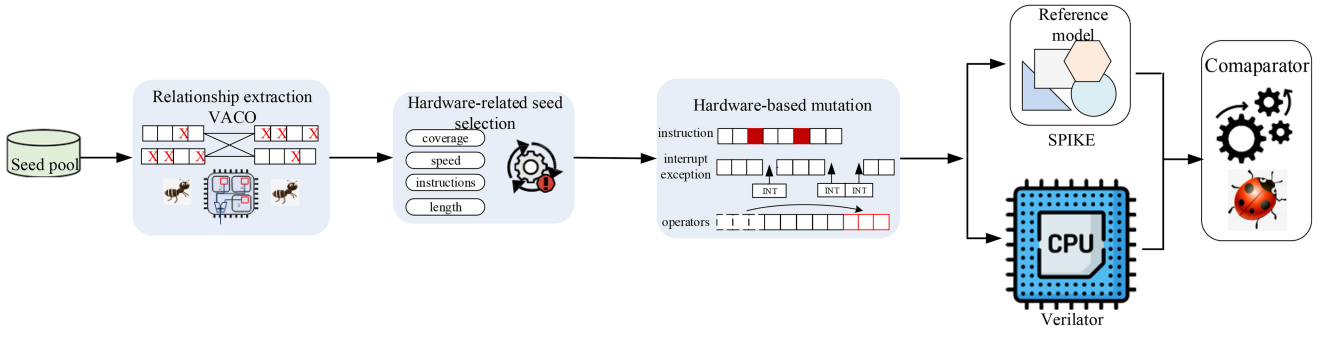


Fig. 3. Overview of the basic procedures in INSTILLER, including VACO, seed selection and mutation, and verilator.

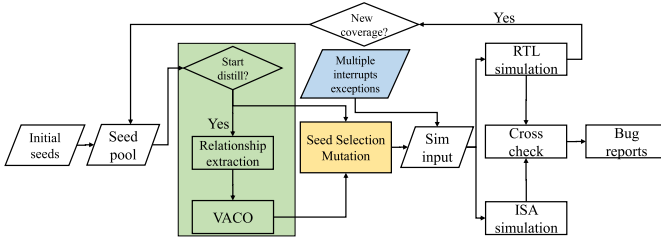


Fig. 4. Fuzzing procedures of INSTILLER, where the colored parts are newly proposed mechanisms compared with traditional fuzzing.

algorithm. The output of distillation is the most effective input and its length for the current fuzzing status. After seed selection and mutation, the input instructions are inserted with multiple interruptions and exceptions, which are ready for execution. ISA simulation and RTL simulation will be executed, and their results are cross-checked to output bug reports. In general, Fig. 4 shows the fuzzing procedure of INSTILLER, and the colored parts are the modification to the basic fuzzing process.

B. Input Instruction Distillation Based on VACO

In state-of-the-art RTL fuzzing work [10], the length of the input instruction keeps increasing as the fuzzing continues. Long input instructions slow down the fuzzing process and are unfriendly for fuzzing. Therefore, we propose input instruction distillation to keep the inputs short and effective. The distillation includes relationship extraction and the VACO algorithm.

Relationship Extraction: In the execution of the CPU, the complex operations are finished by some related instructions and they should be treated as a group. For example, in Fig. 5, the three instructions complete an ADD operation. Therefore, it is rational to extract the relationships between instructions. According to our preliminary study,¹ the relationships between RTL input instructions can be divided into software relationships and hardware relationships. If there are relationships between instructions, we collect them to form instruction groups. These groups are then used in the VACO algorithm.

Software relationships include data-flow and control-flow relationships. First, we sort all the executed input instructions with coverage. Then, beginning from the input instructions with the most coverage, the instructions are traversed. If two instructions share the same registers, there is a data-flow

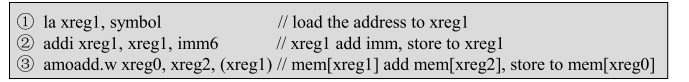


Fig. 5. Example of the relationships between instructions, where the three instructions all operate on the same register.

Algorithm 2 Relationship Extraction Algorithm

Require: Software inputs W_s , hardware inputs W_h , coverage cov

```

1:  $W = \text{sort}(W_s, cov)$ 
2:  $W_D = \emptyset$  /* instruction groups */
3: for  $i \rightarrow W$  do
4:   for  $j \rightarrow W$  do
5:     if  $register_i == register_j$  then
6:        $W_D = W_D + (i, j)$ 
7:     end if
8:     if  $target(i) == j$  then
9:        $W_D = W_D + (i, j)$ 
10:    end if
11:  end for
12: end for
13: for  $i \rightarrow W_h$  do
14:   if  $i \Rightarrow \text{clock, interrupt, privilege, register}$  then
15:      $W_D = W_D + i$ 
16:   end if
17: end for

```

Ensure: Instruction groups W_D

relationship between them. If one input is the jump target of another, there is a control-flow relationship between them. Instructions that have software relationships are collected to form groups.

Hardware relationships include clock cycles, interruptions, privilege levels, and special registers. For example, if inserting an interruption after an instruction can change the privilege level (priority), we consider there is a hardware relationship. Another example is the nonaligned load and store addresses are exceptional. When the hart time comparator (a memory mapping register named `mtimecmp`) is larger than the real-time counter `mtime`, the clock interruption will be triggered. These instructions together with the hardware events are collected to form groups.

Algorithm 2 shows the process of the relationship extraction procedure. This approach outputs the instruction groups to VACO.

The VACO Algorithm: As we discussed above, by simulating the behaviors of the ant colony, ACO is a classic optimization algorithm to find the shortest path between cities. It is an iterative algorithm, and it outputs the best solution when the

¹This study is conducted by investigating the RISC-V Instruction Set Manual Volume I and II [21], [22].

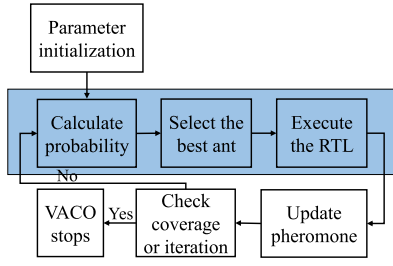


Fig. 6. Basic workflow of VACO, where the colored parts are newly designed compared with ACO.

iteration is done. To adopt ACO in our scenario, we need to solve three problems: 1) when to start the algorithm? 2) how to model the factors in fuzzing into the algorithm? and 3) when to stop the process?

1) The algorithm is invoked when there is a continuous average coverage decrease. Average coverage is coverage divided by the input instruction length. When this indicator decreases, the input instruction length is too long, and the input is not effective in finding new coverage.

2) We model the length of the input instructions as the number of ants, and the RTL circuits as the cities in ACO. The scale of RTL² (number of cities) is n , and the current length of input instruction (number of ants) is m . In each iteration, for every ant i and every path j , we calculate the pheromone table as

$$\text{pher}_j = (1 - \rho) * \text{pher}_j + \sum_{i=1}^m \Delta \text{pher}_{j,i}^i. \quad (1)$$

In (1), ρ means the evaporation rate of pheromones, which is a tunable parameter. The definition of $\Delta \text{pher}_{j,i}^i$ is

$$\Delta \text{pher}_{j,i}^i = \begin{cases} \frac{1}{\text{length}_i}, & \text{ant } i \text{ traverses path } j \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Then, a probability table is calculated by

$$p_j^i = \frac{\text{pher}_j * h_j}{\sum_{k \text{ not traversed by } i} \text{pher}_k * h_k}. \quad (3)$$

This equation uses coverage as heuristics (h_j), which is proportional to the coverage of path j . This is where we use the instruction groups. Every instruction group stands for an ant. Executing the instructions in a group is the process of an ant walking through the cities. The best candidate for the next ant is selected with probability table p_j^i . Therefore, by appending the best group in this iteration to the current input instructions, the length of input will increase by one after each iteration of the algorithm. When the algorithm is terminated, the length of the input instructions is the most effective length for the current fuzzing status.

3) The algorithm will iterate until there is an average coverage increase compared with that before starting the algorithm.

As Fig. 6 shows, the basic workflow is an iterative algorithm. Our proposed algorithm is a variant of the classic ACO. The colored parts in the figure are the major differences between VACO and ACO.

²In this article, we use the coverage definition of DiFuzzRTL. Therefore, the scale of RTL means the number of control registers [10].

Algorithm 3 VACO Algorithm

Require: Number of iterations Max_iter , evaporation rate ρ , RTL scale n , input length m , group W_D

```

1: while iteration < Max_iter do
2:   pher[:n] = [1...1]
3:   heuristics[:iteration] = coverage
4:   p = Equation (3)
5:   id = roulette_wheel_selection(p)
6:   coverage' = exe(id) /* Executing this input instruction results
   in coverage' */
7:   if average coverage increased then
8:     break
9:   end if
10:  pher = (1 - ρ) * pher + ΣΔpher
11: end while
12: m = m + len(id)

```

Ensure: The most effective input instruction length m

- 1) The number of ants in ACO is constant, while it is variable in VACO. When the algorithm is terminated, the generated length is the best input instruction length.
- 2) In VACO, the probability calculation is based on the coverage of the fuzzing process. However, the transition probability in ACO is related to the distance between cities.
- 3) The classic ACO does not contain the process of executing the ant in the RTL. Therefore, introducing the execution into VACO inevitably causes performance overhead. We integrate the RTL execution in VACO into the main fuzzing loop to reduce the performance overhead.

In conclusion, Algorithm 3 shows the process of the VACO algorithm. By receiving the parameters, VACO iterates and finally outputs the best input instruction and length in the current fuzzing status. The length is shorter and the input instruction is more effective in finding new coverage. Therefore, the input distillation is finished. In line 5, the roulette wheel is a classic selection algorithm [26].

C. Realistic Interruption and Exception Handling

First, previous work has no exception inserted in the input instructions. We include exceptions in the RTL inputs to solve this problem. Next, in real-world applications, multiple interruptions and exceptions are common during CPU execution. We consider this situation in RTL testing and enable multiple interruptions and exceptions together with their respective handling strategies. In our design, multiple interruptions and exceptions are inserted in the input instructions for simulation. Every interruption and exception is analyzed to extract the relative information, e.g., the address of the interruption and the cause of the exception.

Fig. 1 shows the difference between multiple interruptions and exceptions and single ones. Several instructions `inst n` constitute a complete input. Interruptions and exceptions can be inserted between the instructions. The input on the right with multiple interruptions or exceptions is the one in INSTILLER. This input can better simulate real-world CPU execution.

Moreover, interruptions and exceptions have specific priorities. For example, in RISC-V, the priority of interruptions in machine mode is higher than that in supervisor mode [22]. We consider different interruptions and exceptions with different

priorities in fuzzing. In a situation where interruptions and exceptions with higher priority are triggered when a lower-priority one is being handled, more RTL states can be covered than testing without priorities.

In conclusion, in the process of fuzzing the RTL, we include both multiple interruptions and exceptions along with their priorities to test the target RTL more thoroughly.

D. Seed Selection

Previous work in RTL fuzzing ignores the importance of seed selection and hardware features in improving the performance of fuzzing. In seed selection, we focus on the normalized heuristics of input instructions to make decisions. Normalized heuristics means the heuristics score of an input divided by its length. The reason for using normalized heuristics is that one important perspective of our work is to distill seeds and generate shorter input instructions. A higher normalized heuristics score indicates the input instruction is more potential for fuzzing, and its length is relatively short at the same time.

In INSTILLER, we use the following heuristics to select seeds.

- 1) *Basic Heuristics*: It includes coverage increase (*cov*) and execution speed (*speed*). These metrics of heuristics are commonly seen in fuzzing tools [27], [28], [29]. We use these metrics to score seeds in this article. Specifically, coverage is the most important metric in coverage-guided fuzzing. Therefore, it is used in our tool. Then, execution speed is also a crucial factor in fuzzing [11], [12], [13], [14], [15], [16], [55], and we integrate this metric in our heuristics.
- 2) *RTL Heuristics*: This category contains metrics related to RTL hardware, including the number of load or store instructions (*ld_st*), floating point instructions (*fp*), and jump instructions (*jp*). According to our study of recent bugs in real-world cores [30], [31], e.g., Boom and Rocket, we find out that these instructions are the cause of multiple bugs. Therefore, we use these metrics in scoring the input instructions.

In conclusion, our heuristics calculation can be summarized as

$$\text{heuristics} = \frac{\omega * \text{cov} * \text{speed} + \text{ld_st} * \text{fp} * \text{jp}}{\text{len}}. \quad (4)$$

In this equation, ω is the proportion of basic metrics in the heuristics, which controls the weight between basic and RTL metrics. Based on this heuristics, we select input instructions with the highest score in the seed pool.

E. Mutation

In the design of DiFuzzRTL, the mutator can only add instructions to the inputs in mutation [10]. In other words, the mutation operation will keep the length of input instructions increasing. However, in common fuzzing tools, such as AFL, it is widely acknowledged that fuzzers should be equipped with various effective mutation strategies [23], [32], [33], e.g., “dictionary” (replacing part of the input instructions with tokens)³ and “splice” (splicing two inputs to generate one) in AFL.

³Token means an item in the dictionary.

We use several mutation strategies in INSTILLER to improve fuzzing.

- 1) *The Mutation Strategy of DiFuzzRTL*: This article indicates its strategy is effective in fuzzing [10]. Therefore, we retain it in INSTILLER.
- 2) *Dictionary*: In input instruction distillation, we get the distilled inputs, in which we can extract dictionary tokens to guide mutation. These tokens are effective in fuzzing and short in length. By replacing part of the original input instruction with these tokens, this “dictionary” mutation strategy is completed.
- 3) *Insertion*: We randomly insert new instructions to the inputs.
- 4) *Deletion*: Part of the input instruction is deleted, and the length of the input will decrease.

These four types of mutation strategies are similar to the mutation in binary fuzzing, but they are different. The input instruction of binary fuzzing can be encoded to sequences of “1”s and “0”s. The mutation on the sequence can be treated as mutating a simple string to enumerate all cases. Therefore, “bitflip” (flipping “1” to “0,” or “0” to “1”) in AFL is highly effective in discovering new paths [32]. However, if there are illegal instructions, randomly mutating the input instructions of RTL can be often meaningless. Therefore, we use several techniques in the design of mutation strategies. For example, when inserting instructions, we use previously used ones with higher probability and newly generated ones with lower probability. Another example is in deletion, when deleting one instruction, the related instruction will also be deleted, e.g., a jump instruction and its target.

Furthermore, these four types of mutation in INSTILLER are invoked in different situations. They are chosen with different probabilities, which are calculated by weighted metrics. Every strategy is chosen with a different probability. If the condition is satisfied, the respective mutation strategy is selected. Otherwise, INSTILLER uses the original mutation. This can be concluded as

$$\text{mutation} = \begin{cases} \text{dictionary, coverage decreases} \\ \text{insertion, len} < l \\ \text{deletion, len} > l \\ \text{basic, otherwise.} \end{cases} \quad (5)$$

IV. IMPLEMENTATION

We implement INSTILLER based on DiFuzzRTL. In general, the RTL instrumentation is finished with FIRRTL. We use Spike for ISA simulation, and cocotb for RTL simulation. The main fuzzing loop is implemented in Python.

In total, the implementation of INSTILLER can be divided into three parts.

- 1) The input instruction distillation process, which includes relationship extraction and VACO. We add this part in *Fuzzer.py*, and there are about 500 lines of code in total. Additionally, we modify the main fuzzing loop to interact with VACO, including the starting and terminating conditions.
- 2) *Interruptions and Exceptions*: We integrate multiple interruptions and exceptions into INSTILLER, as well as their priorities. *signature_checker.py* is modified, where we focus on the special registers for the interruption and exception handling, e.g., *scause* and *mcause*. This part contains about 550 lines of code.

TABLE I
INFORMATION OF THE TARGET CPUs, INCLUDING ISA, NUMBER OF PIPELINES, INSTRUCTION WIDTH, AND RELEASE TIME

Targets	ISA	Pipeline	Width	Year
mor1kx	OpenRISC	6-stage	32-bit	2013
or1200	OpenRISC	5-stage	32-bit	2000
Boom	RISC-V	4-stage	32-bit	2017
Rocket	RISC-V	5-stage	32-bit	2016

3) *Seed Selection and Mutation Strategies*: We implement the heuristics calculation and four mutation strategies in `mutator.py`, which include about 700 lines of code. Both seed selection and mutation are integrated into the original process, which will cause no additional steps.

V. EVALUATION

In our evaluation, we answer the following research questions.

- 1) *RQ1*: Can INSTILLER increase code coverage and shorten input instruction length?
- 2) *RQ2*: Does INSTILLER have better vulnerability discovery ability than state-of-the-art RTL fuzzers?
- 3) *RQ3*: What is the performance increase in execution speed of INSTILLER?
- 4) *RQ4*: How does the VACO algorithm perform?
- 5) *RQ5*: How do the techniques on interruptions and exceptions perform?
- 6) *RQ6*: How do the seed selection and mutation strategies perform?

A. Setup

Targets: In our experiments, the target RTL designs include mor1kx [34], or1200 [35], Boom [30], and Rocket [31]. These are popular RTL cores, and state-of-the-art papers used them in the experiments [10], [25]. Therefore, including these RTL designs in our evaluation demonstrates persuasiveness and representativeness. The detailed information is listed in Table I.

Compared Tools: To demonstrate the performance of INSTILLER, we compare it with DiFuzzRTL. DiFuzzRTL is one of the state-of-the-art CPU testing tools. We will compare INSTILLER with DiFuzzRTL in different aspects such as coverage.⁴

Metrics: We use coverage, the length of input instructions, the number of mismatches, and execution speed as the metrics in our evaluation. For every target CPU, we repeat the 24-h fuzzing 10 times. In addition, we calculate the p values and $\hat{A}_{12}$ values of all the experiment results to eliminate the effect of randomness in fuzzing [36]. If the $p < 0.05$ and $\hat{A}_{12} > 0.5$, then this specific comparison shows a statistically significant difference.

B. Evaluation on Coverage

Table II is the coverage results of INSTILLER and DiFuzzRTL. According to the table, INSTILLER outperforms DiFuzzRTL in all the targets, and all the coverage increase is more than 11%. In all the comparisons with DiFuzzRTL, the

⁴The source code of TheHuzz [25] is not available. Therefore, it is not evaluated in this article.

TABLE II
EVALUATION ON COVERAGE OF INSTILLER AND DiFUZZRTL, WHERE THE VALUE IN THE BRACKET DENOTES THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

Targets	INSTILLER	DiFuzzRTL	p value	$\hat{A}_{12}$
mor1kx	201288.7(+32.6%)	151840.5	5.11×10^{-4}	1.0
or1200	269504.8(+36.5%)	197503.9	6.84×10^{-5}	1.0
Boom	547433.5(+28.8%)	425183.7	9.13×10^{-5}	1.0
Rocket	101489.6(+11.9%)	90715.3	8.98×10^{-5}	1.0
Average	279929.15(+29.4%)	216310.85	1.90×10^{-4}	1.0

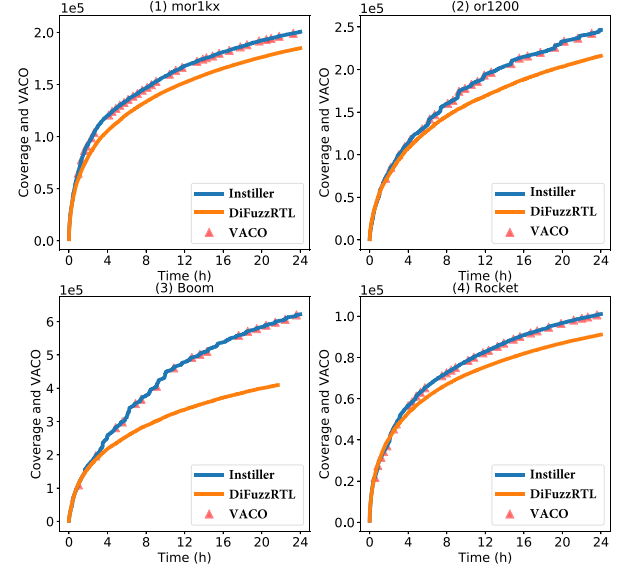


Fig. 7. Coverage and the effectiveness of VACO over time, where the x-axis is the time, and the y-axis is the coverage.

p values are less than 0.05, and the $\hat{A}_{12}$ values are greater than 0.5. The results indicate statistically significant differences. In the comparison in or1200, INSTILLER has 36.5% more coverage than DiFuzzRTL, which is the greatest difference among all the results. On average, INSTILLER reaches 29.4% more coverage, and this result also has a statistically significant difference.

Fig. 7 shows the coverage growing over time of INSTILLER and DiFuzzRTL. We record the results within 24 h of fuzzing. According to the figure, in all the targets, the coverage of INSTILLER is greater, and the growth of coverage in INSTILLER is faster than DiFuzzRTL.

The results in Table II and Fig. 7 demonstrate the effectiveness of input instruction distillation. The internal reason is that we distill inputs based on the coverage performance of input instructions, e.g., line 1 in Algorithm 2 sorts the inputs based on coverage.

In conclusion, according to these experiment results, INSTILLER has better coverage exploration ability than DiFuzzRTL.

C. Evaluation on Input Instruction Length

Table III shows the input instruction length during fuzzing of INSTILLER and DiFuzzRTL, respectively. In all the target CPU cores, INSTILLER shortens the length of input instructions, and all the decrease is more than 74%. Especially in the Rocket core, the decrease of length reaches the maximum of

TABLE III

EVALUATION ON INPUT INSTRUCTION LENGTH OF INSTILLER AND DiFuzzRTL, WHERE THE VALUE IN THE BRACKET DENOTES THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

Targets	INSTILLER	DiFuzzRTL	p value	$\hat{A}_{12}$
mor1kx	451.39(-77.6%)	2018.21	9.56×10^{-5}	1.0
or1200	385.63(-80.6%)	1986.70	7.55×10^{-5}	1.0
Boom	486.75(-74.1%)	1882.80	9.13×10^{-5}	1.0
Rocket	420.67(-83.6%)	2557.67	8.98×10^{-5}	1.0
Average	436.11(-79.3%)	2111.34	8.80×10^{-5}	1.0

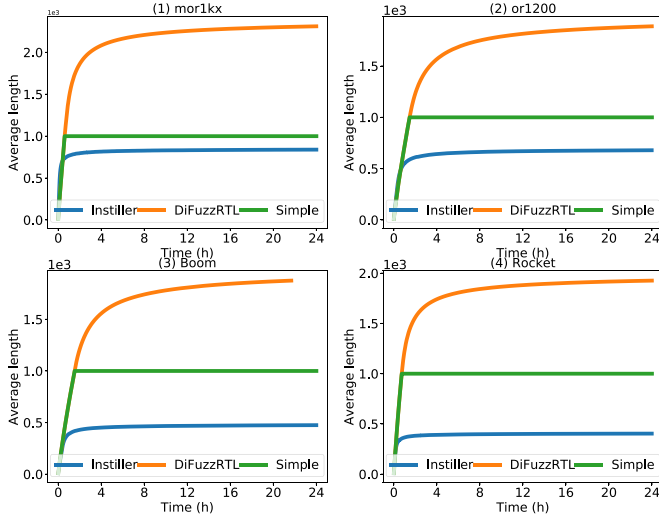


Fig. 8. Input instruction length over time, where the x -axis is the time, and the y -axis is the average length.

83.6%. In addition, all the comparisons with DiFuzzRTL have statistically significant differences. The average input instruction length of INSTILLER is 79.3% shorter than DiFuzzRTL. These results indicate the effectiveness of VACO, which significantly shortens the input instruction length.

Fig. 8 is the result of input instruction length over time. In the 24 h, compared with DiFuzzRTL, INSTILLER shortens the length.

The results in this section show a significant difference between input instruction length in INSTILLER and DiFuzzRTL. Decreasing the input length is the main focus of this work, and the mechanism in VACO contributes to the results. Moreover, the results of DiFuzzRTL in Table II and Fig. 8 demonstrate the statement in Section I that coverage does not increase proportionally to input length.

Therefore, based on the results of coverage and input instruction length, we can answer RQ1: INSTILLER can increase code coverage and shorten input instruction length at the same time.

D. Evaluation on Vulnerability Detection

Table IV shows the number of mismatches of differential testing. In the design of INSTILLER and DiFuzzRTL, if the output of ISA simulation is different from RTL execution, i.e., a mismatch, a potential bug is detected. It is rational to use the number of mismatches to demonstrate the vulnerability detection ability of the fuzzing tools. In the table, except for mor1kx, INSTILLER outperforms DiFuzzRTL in all the

TABLE IV

NUMBER OF MISMATCHES OF INSTILLER AND DiFuzzRTL, WHERE THE VALUE IN THE BRACKET DENOTES THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

Targets	INSTILLER	DiFuzzRTL	p value	$\hat{A}_{12}$
mor1kx	110.1(-3.9%)	120.9	0.98	0.2
or1200	598.3(+6.7%)	560.9	0.001	1.0
Boom	5546.0(+18.8%)	4666.6	0.01	1.0
Rocket	33.3(+14.0%)	29.2	0.06	0.7
Average	1573.9(+17.0%)	1344.4	0.26	0.73

TABLE V

EXECUTION SPEED PER SECOND OF INSTILLER AND DiFuzzRTL, WHERE THE VALUES IN THE BRACKET DENOTE THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

Targets	INSTILLER	DiFuzzRTL	p value	$\hat{A}_{12}$
mor1kx	0.27(+8.0%)	0.25	0.99	0.15
or1200	0.40(+5.3%)	0.38	0.98	0.21
Boom	0.25(+4.1%)	0.24	0.99	0.10
Rocket	0.36(+9.1%)	0.33	0.99	0.20
Average	0.32(+6.7%)	0.30	0.99	0.17

targets. The greatest improvement is in Boom core, with an increase of 18.8%. On average, INSTILLER also outperforms DiFuzzRTL in vulnerability discovery, and it detects 16.9% more mismatches.

Furthermore, we manually investigate why DiFuzzRTL outperforms INSTILLER in mor1kx. In Table II, INSTILLER has more coverage. Our investigation shows the fuzzers cover different parts of mor1kx. Moreover, by looking into the code of mor1kx, we find out that the hardware circuits covered by INSTILLER cannot trigger as many mismatches as DiFuzzRTL. These factors lead to the results in Table IV.

Therefore, we can answer RQ2: INSTILLER has better vulnerability detection ability than DiFuzzRTL.

E. Evaluation on Execution Speed

Table V shows the execution speed of INSTILLER and DiFuzzRTL. Execution speed is the result of dividing the number of executions by the time. In general, INSTILLER runs faster than DiFuzzRTL. The greatest difference is in Rocket, which is 9.1%. On average, the performance increase in INSTILLER is 6.7%, which shows the effectiveness of distilled input instructions. Shorter inputs require fewer CPU cycles to execute.

In Section V-C, the input instruction length of INSTILLER is 79.3% shorter on average. Intuitively, the performance increase should have been more than 6.7%. We investigate the cause of this result. First, there is no direct correspondence between the input length and execution speed of fuzzing. Having 79.3% short length does not mean executing 79.3% faster. Second, there is a performance overhead in the process of relationship extraction and VACO. The relationship extraction enumerates all the executed input instructions, and VACO is also an iterative process. Therefore, the performance overhead is unavoidable to finish these processes.

In addition, we conduct extra experiments to show their specific overhead. Table VI shows the respective average speed of four different configurations of INSTILLER. When relationship extraction is disabled, the fuzzer can run 6.3% faster than the original INSTILLER. However, investigating

TABLE VI

PERFORMANCE OVERHEAD OF RELATIONSHIP EXTRACTION AND VACO, WHERE THE VALUES IN THE BRACKET DENOTE THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

	INSTILLER	INSTILLER ^{-R}	INSTILLER ^{-V}	INSTILLER ^{-RV}
Speed	0.32	0.34(+6.3%)	0.29(-9.4%)	0.30(-6.3%)

¹ INSTILLER^{-R} denotes INSTILLER without relationship extraction.

² INSTILLER^{-V} denotes INSTILLER without VACO.

³ INSTILLER^{-RV} denotes INSTILLER without relationship extraction and VACO.

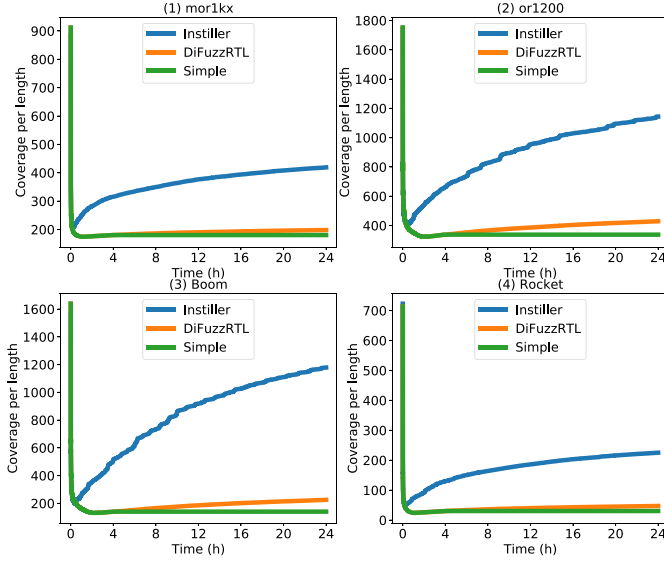


Fig. 9. Coverage divided by input instruction length over time, where the x-axis denotes time, and the y-axis denotes coverage divided by length.

coverage data shows that this configuration has less coverage than INSTILLER. Therefore, disabling relationship extraction has an effect on speed and coverage. If there is no VACO in INSTILLER, the execution speed decreases by 9.4%. The reason is that disabling VACO means input distillation is disabled, which cannot utilize shorter input instructions. Besides, the execution of relationship extraction in INSTILLER^{-V} has overhead. The average speed of INSTILLER^{-RV} is the same as DiFuzzRTL in Table V, which means other approaches of INSTILLER, e.g., seed selection and mutation, cause negligible performance overhead.

We can answer RQ3: The performance increase of INSTILLER is 6.7% on average.

F. Results of VACO

Fig. 9 shows the results of coverage divided by input instruction length over time of INSTILLER and DiFuzzRTL. Both of the lines experience a high start in the figure. The reason is that the first input instruction length is always “1,” and executing the first input brings about a 700 to 1800 coverage increase. Regardless of the high start, INSTILLER has higher coverage per length than DiFuzzRTL. This result indicates the input instructions in INSTILLER are more effective in discovering new coverage. The mechanisms of relationship extraction and VACO ensure this result.

Fig. 7 shows the coverage results of INSTILLER and DiFuzzRTL, and how VACO affects coverage in the 24 h. The red circles in the figure indicate the enabling of VACO. Every

TABLE VII

RESULTS OF COVERAGE AND INPUT INSTRUCTION LENGTH OF DIFFERENT CONFIGURATIONS OF INSTILLER, WHERE THE VALUES IN THE BRACKET DENOTE THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

	INSTILLER	INSTILLER ^{-R}	INSTILLER ^{-V}	INSTILLER ^{-RV}
Coverage	279929.2	248663.5(-11.2%)	255439.2(-8.7%)	217197.5(-22.4%)
Length	436.1	498.4(+14.3%)	2185.9(+401.2%)	2254.8(+417.0%)

¹ INSTILLER^{-R} denotes INSTILLER without relationship extraction.

² INSTILLER^{-V} denotes INSTILLER without VACO.

³ INSTILLER^{-RV} denotes INSTILLER without relationship extraction and VACO.

time VACO is enabled, the coverage increases more rapidly than DiFuzzRTL.

Table VII shows the results of coverage and input instruction length of four different configurations of INSTILLER. For coverage, disabling relationship extraction and VACO will have a negative effect. With a 22.4% decrease, INSTILLER^{-RV} has the least coverage, which is almost the same as the result of DiFuzzRTL in Table II. For input instruction length, the VACO algorithm has the most effect. Disabling VACO alone causes a 417.0% length increase, which is also close to the result of DiFuzzRTL in Table III.

Besides, we compare VACO with a simpler method which sets an upper limit for the length and discards seeds exceeding this limit.⁵ The results are shown in Figs. 8 and 9. The upper limit is set to 1000 in our evaluation. Fig. 8 denotes the average length over time of this simple strategy. In this figure, the average length of Simple grows as the fuzzing process continues. When the length reaches the limit, it stays unchanged until the end of fuzzing.

Moreover, as shown in Fig. 9, the result of Simple experiences a high start in the beginning, which is similar to INSTILLER and DiFuzzRTL. However, as the fuzzing process continues, the coverage per length of Simple stays at a low value compared with INSTILLER, and the value remains unchanged several hours after the start of fuzzing. Though simply setting an upper limit for the length and discarding seeds exceeding this limit can control the length of the input sequence, there is no benefit in improving coverage. Our VACO algorithm surpasses the basic upper-bound-limiting strategy by controlling the input length and increasing coverage at the same time.

Furthermore, we conduct experiments comparing the bug-finding performance between INSTILLER and INSTILLER without VACO (denoted as INSTILLER^{-V}). As shown in Table VIII, INSTILLER outperforms INSTILLER^{-V} in all the target CPUs. On average, INSTILLER has 12.5% more mismatches, demonstrating better vulnerability discovery ability of INSTILLER. The reason behind these results is mainly due to the coverage of the fuzzing process. As shown in Table VII, INSTILLER^{-V} has 8.7% less coverage than INSTILLER. Covering more code ensures INSTILLER to find more bugs.

Therefore, we can answer RQ4: Experiment results show the effectiveness of VACO, together with relationship extraction, which can increase code coverage, shorten input instruction length, and find more bugs at the same time.

⁵“Simple” strategy is the version of INSTILLER that replaces VACO with an upper-bound-limiting strategy.

TABLE VIII
NUMBER OF MISMATCHES OF INSTILLER AND INSTILLER WITHOUT VACO (DENOTED AS INSTILLER^{-V}), WHERE THE VALUES IN THE BRACKET DENOTE THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

Targets	INSTILLER	INSTILLER ^{-V}	p value	$\hat{A}_{12}$
morlxx	110.1(+7.4%)	102.5	0.04	0.7
or1200	598.3(+6.5%)	561.7	0.002	1.0
Boom	5546.0(+13.3%)	4897.1	0.01	1.0
Rocket	33.3(+10.3%)	30.2	0.045	1.0
Average	1573.9(+12.5%)	1397.9	0.022	0.93

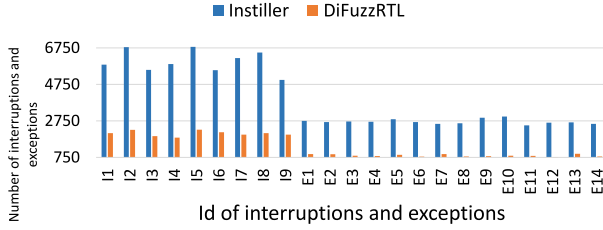


Fig. 10. Number of interruptions and exceptions of INSTILLER and DiFuzzRTL, where the x-axis denotes ID, and the y-axis denotes the number.

G. Results of Multiple Interruptions and Exceptions

We collect nine interruptions (I1–I9) and 14 exceptions (E1–E14) from [22]. In this part of the evaluation, we insert interruptions and exceptions into the input instructions with 50% probability. Moreover, we set the limit of interrupts and exceptions to three, respectively, to show the effectiveness of multiple ones. However, this does not mean INSTILLER can only support three interruptions or exceptions. The limit can be configured as needed.

Fig. 10 shows the number of interruptions and exceptions of INSTILLER and DiFuzzRTL. The number of interruptions and exceptions of INSTILLER are more than DiFuzzRTL. Multiple ones are inserted into the input instructions, and the chance of triggering new states and bugs is higher in INSTILLER.

In addition, we compare INSTILLER with DiFuzzRTL by considering the priorities of interruptions and exceptions. In the design of RISC-V, the interruptions and exceptions have fixed priorities. For example, “I2” is higher than “I0,” and “E3” is higher than “E0.” Therefore, different combinations of them indicate different combinations of priorities. In this part, we define interruption state transition (IST) and exception state transition (EST) to describe different combinations of interruptions and exceptions. For example, there are “I3” and “I7,” and “E2” in an input instruction. The IST of this input instruction is 1 ((3 << 1) XOR 7), and EST is 4 ((2 << 1) XOR 0). Here, we use $(ID_1 \ll 1) \text{ XOR } ID_2$ as a hash method to represent IST or EST. In our definition, the more different values of IST or EST, the more different states a fuzzer can reach.

Fig. 11 shows the distribution of state transitions of interruptions and exceptions. The IST of INSTILLER ranges from 0 to 64, while that of DiFuzzRTL ranges from 2 to 18. The EST result is similar. The states INSTILLER can reach are more than DiFuzzRTL. This result indicates that by considering multiple interruptions and exceptions with their combinations, INSTILLER can trigger more states in fuzzing, and therefore, it has a higher chance to discover bugs. Note that in the figure, the result of “DiFuzzRTL” is covered by “Both,” so we cannot see the result of it.

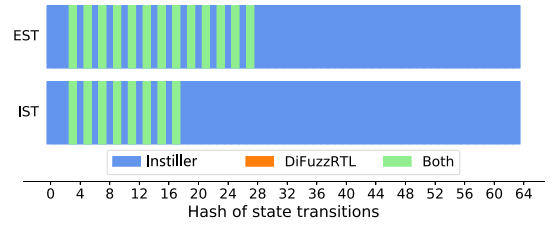


Fig. 11. Distribution of IST and EST of INSTILLER and DiFuzzRTL.

TABLE IX
NUMBER OF MISMATCHES OF INSTILLER AND INSTILLER WITHOUT MULTIPLE INTERRUPTIONS AND EXCEPTIONS (DENOTED AS INSTILLER^{-IE}), WHERE THE VALUES IN THE BRACKET DENOTE THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

Targets	INSTILLER	INSTILLER ^{-IE}	p value	$\hat{A}_{12}$
morlxx	110.1(+2.8%)	107.1	0.015	0.85
or1200	598.3(+3.7%)	577.2	0.01	0.9
Boom	5546.0(+2.5%)	5411.8	0.03	1.0
Rocket	33.3(+7.1%)	31.1	0.02	1.0
Average	1573.9(+2.6%)	1531.8	0.019	0.94

TABLE X
COVERAGE AND LENGTH COMPARISON OF INSTILLER WITH OTHER CONFIGURATIONS, WHERE THE VALUES IN THE BRACKET DENOTE THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

Targets	INSTILLER	INSTILLER ^{-S}	INSTILLER ^{-M}	INSTILLER ^{-SM}
Coverage	279929.15	267754.12(-4.3%)	259643.88(-7.2%)	255386.9(-8.8%)
Length	436.11	467.08(+7.1%)	455.86(+4.5%)	478.1(+9.6%)

¹ INSTILLER^{-S} denotes INSTILLER without seed selection strategy.

² INSTILLER^{-M} denotes INSTILLER without mutation strategy.

³ INSTILLER^{-SM} denotes INSTILLER without seed selection and mutation strategies.

Moreover, Table IX compares the bug-finding performance between INSTILLER and INSTILLER without multiple interruptions and exceptions (denoted as INSTILLER^{-IE}). As the table illustrates, INSTILLER surpasses INSTILLER^{-IE} in all the CPUs, reaching up to 7.1% in Rocket. On average, INSTILLER also outperforms the competitor by 2.6%. Besides, all the p values are less than 0.05, and all the $\hat{A}_{12}$ values are greater than 0.5, indicating all the results have significant differences.

We investigate the source code and find out that some mismatches in Table IX reside in the PLIC, nested interruption handling, and exception handling of CPU implementations, as listed in Section II-A and Fig. 1. For example, we witness a store page fault exception handling in Rocket core using INSTILLER, while we cannot reproduce it with other fuzzers or INSTILLER^{-IE}. This result verifies our investigation in Section II-A that enabling multiple interruptions and exceptions is effective to CPU fuzzing.

Therefore, we can answer RQ5: The techniques of multiple interruptions and exceptions are effective.

H. Effectiveness of Seed Selection and Mutation

Table X shows the results of coverage and length of different configurations of INSTILLER. By using (4), the fuzzing process is guided toward increasing coverage. Therefore, INSTILLER has 4.3% more coverage than INSTILLER^{-S}. Similarly, (5) uses dictionary mutation to mitigate coverage decrease, and INSTILLER^{-M} has 7.2% less coverage when the

TABLE XI

NUMBER OF MISMATCHES OF INSTILLER AND INSTILLER WITHOUT SEED SELECTION AND MUTATION (DENOTED AS INSTILLER^{-SM}), WHERE THE VALUES IN THE BRACKET DENOTE THE INCREASE OR DECREASE COMPARED WITH THE COMPETITORS

Targets	INSTILLER	INSTILLER ^{-SM}	p value	$\hat{A}_{12}$
mor1kx	110.1(+5.6%)	104.3	0.002	1.0
or1200	598.3(+5.1%)	569.3	0.01	1.0
Boom	5546.0(+2.7%)	5399.3	0.001	1.0
Rocket	33.3(+7.8%)	30.9	0.015	1.0
Average	1573.9(+3.0%)	1525.95	0.007	1.0

mutation strategy is disabled. The coverage of INSTILLER^{-SM} is more than DiFuzzRTL in Table II. This result indicates the effectiveness of other techniques in INSTILLER, e.g., the VACO algorithm.

For input instruction length, the result of INSTILLER^{-S} is 7.1% greater than INSTILLER. In (4), we use heuristics divided by length, aiming to select the shortest input instruction. Therefore, the seed selection strategy also helps shorten input instruction length. In our mutation strategy, the length of the input instruction is controlled by insertion and deletion. The result of INSTILLER^{-M} is 4.5% greater than INSTILLER, indicating the effectiveness of the mutation strategy. INSTILLER^{-SM} has a shorter length than DiFuzzRTL in Table III, proving the huge effect of distillation on input instruction length.

Moreover, we conduct experiments to compare INSTILLER with INSTILLER without seed selection and mutation (denoted as INSTILLER^{-SM}), which is illustrated in Table XI. INSTILLER outperforms INSTILLER^{-SM} in all the tested CPUs. Especially in Rocket, the performance difference reaches the maximum of 7.8%. On average, INSTILLER surpasses INSTILLER^{-SM} by 3.0%, and all the differences are significant according to the p values and $\hat{A}_{12}$ values. Referring to (4) and (5), we add cov in the heuristics in seed selection and mutation. Covering more parts of the code contributes to the bug-finding performance of INSTILLER.

Therefore, we can answer RQ6: The seed selection and mutation strategies are also effective in increasing coverage, shortening input length, and detecting bugs.

I. Comparison With Other Hardware Fuzzers

Despite using differential testing, there are other types of hardware fuzzers. In this part, we compare INSTILLER with Hardware Fuzzing Pipeline (HFP) [37] and RFuzz [9]. Both HFP and RFuzz detect bugs without differential testing. There are eight experiment targets, including FFTSmall, Sodor1Stage, Sodor3Stage, Sodor5Stage, TL12C, TLPWM, TLSPI, and TLUART. These targets are collected from the experiments of HFP and RFuzz. We use HDL line coverage as the coverage metric to show the results. As Fig. 12 shows, INSTILLER has more HDL line coverage than HFP and RFuzz. The internal reason for this result is that we propose instruction distillation based on coverage in INSTILLER. The distilled input instructions are short in length and beneficial to coverage. Results in Section V-B have similar conclusions that INSTILLER has more coverage than DiFuzzRTL.

J. Real-World Vulnerabilities

To prove the ability to detect real-world bugs of INSTILLER, we conduct the following evaluation. We collect 12 real-world

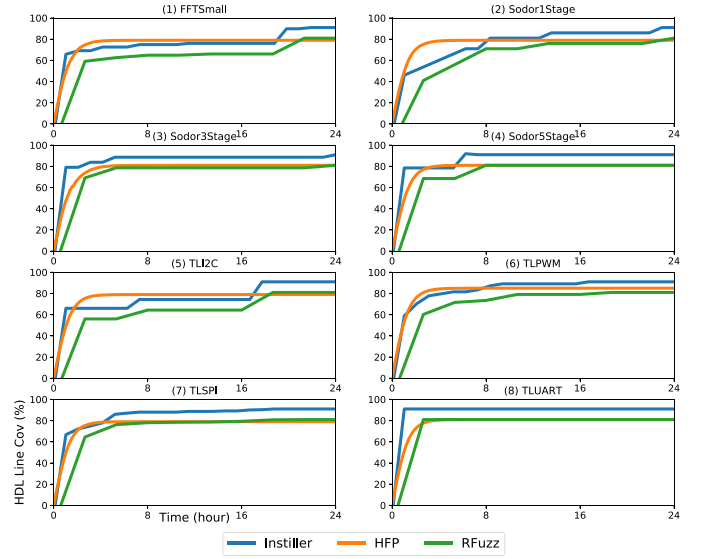


Fig. 12. HDL line coverage of INSTILLER, HFP, and RFuzz over time, where the x-axis denotes time, and the y-axis denotes coverage.

TABLE XII
INFORMATION OF REAL-WORLD BUGS AND THE
DETECTION ABILITY OF INSTILLER

Targets	ID	Description	Reproducible
Boom	V1	Instruction count is inaccurate when minstret is written by software.	✓
Boom	V2	Static rounding is ignored for fdiv.s and fsqrt.s.	✓
Boom	V3	Floating point instruction which has invalid rm field does not raise exception.	✓
Boom	V4	FS bits in mstatus register is set after fle.d instruction.	✗
mor1kx	V5	Incorrect implementation of the carry flag generation.	✗
mor1kx	V6	Missing access checking for privileged register.	✓
mor1kx	V7	ear register not saving instruction virtual address when illegal instruction exception.	✓
mor1kx	V8	l.fll, l.fll instruction decoding fails.	✓
or1200	V9	Incorrect forwarding logic for the GPR0.	✗
or1200	V10	Incomplete update logic of overflow bit formsb & mac instructions.	✓
or1200	V11	Incorrect generation of overflow flag.	✓
Rocket	V12	EBREAK does not increase instruction count.	✗

bugs from the GitHub issue pages of the tested CPUs [30], [31], [34], [35]. As shown in Table XII, the target CPU, bug ID, description, and reproductivity of INSTILLER are listed. In total, INSTILLER can reproduce 8 out of the 12 real-world bugs. All of the detected bugs are produced by the bug-reporting mechanism of INSTILLER, and then they are confirmed by practitioners. This result demonstrates that INSTILLER has the ability to detect real-world vulnerabilities.

VI. DISCUSSION

A. Golden Reference Models

INSTILLER and other hardware fuzzers [10], [25] rely on golden reference models (GRMs) to detect hardware bugs. The validation of many commercial CPUs largely depends on the availability of GRMs. There are many industrial large-scale simulators using GRMs, such as Intel x86 Archsim, AMD x86 Simnow, and ARM Cortex Neoverse. Thus, the availability of GRM is not a constraint for INSTILLER. Moreover, the GRM itself is highly unlikely buggy. They have been carefully designed, which are developed with strict version control and have undergone extensive testing. Therefore, both

TABLE XIII
EVALUATION ON PARAMETER ρ IN (1) ABOUT
COVERAGE, SPEED, AND MISMATCHES

ρ	0.02	0.1	0.5
Coverage	2265493.18	261107.05	279929.15
Speed	0.27	0.26	0.27
Mismatches	1501.8	1511.6	1571.9

TABLE XIV
EVALUATION ON PARAMETER w IN (4) ABOUT
COVERAGE, SPEED, AND MISMATCHES

w	0.1	0.5	1.0	2.0	10.0
Coverage	254329.35	257754.12	268953.90	279929.15	281876.22
Speed	0.24	0.25	0.25	0.27	0.31
Mismatches	1655.9	1641.5	1590.1	1571.9	1105.8

in availability and validity, using GRMs in INSTILLER to detect bugs is not a concern.

B. Different Coverage Metrics

In this article, we use control register coverage mentioned in [10]. However, there are other coverage metrics in hardware fuzzing. In RFuzz [9], mux control coverage is utilized. TheHuzz [25] considers multiple coverage metrics, including branch coverage, condition coverage, FSM coverage, etc. Moreover, software coverage is directly used on the translated model of the hardware RTL in [37]. Although coverage metrics of hardware fuzzing are not the research scope of this article, we still conducted preliminary evaluations on the metrics. Generally speaking, control register coverage performs the best among the metrics in the evaluation. Therefore, we utilize it in this article.

C. Hyperparameters

There are three hyperparameters in the design of INSTILLER, including the tunable parameter in (1), the proportion of basic metrics in (4), and the probability of each mutation operator in (5). For (1), the parameter ρ will affect the strength of mutual influence between the ants, which is related to the global search ability and convergence speed of the algorithm. We compare the three recommended configurations in [38], including 0.02, 0.1, and 0.5. Table XIII shows the comparison of different configurations of ρ . The execution speed of these configurations is similar, and 0.5 reaches the maximum in coverage and mismatches. Therefore, we choose 0.5 as the configuration of ρ in (1).

Moreover, we conduct experiments to discuss the parameter w in (4). This parameter controls the balance between the basic metrics and the RTL metrics. The basic metrics are related to coverage and execution speed, and the RTL metrics are related to CPU bugs.

Table XIV shows how w in (4) influences the results of coverage, execution speed, and the number of mismatches. As w increases, coverage and speed also increase, and the number of mismatches decreases. We select 2.0 as the configuration of w . This value keeps the balance between coverage, speed, and the number of mismatches.

Besides, we conduct experiments to discuss the length parameter l that determines the choices of the mutation operators in (5). If the length of the current fuzzing exceeds l , the “deletion” operator will be chosen. Otherwise, “insertion”

TABLE XV
EVALUATION ON PARAMETER l IN (5) ABOUT
COVERAGE, SPEED, AND MISMATCHES

l	100	400	1000
Coverage	235617.60	279929.15	251265.47
Speed	0.33	0.27	0.20
Mismatches	1211.7	1571.9	1562.3

is selected. Table XV shows the comparison of three different configurations of l . This parameter can affect the input length of fuzzing. We choose 400 as the configuration in our implementation, as it keeps a balance between coverage, speed, and mismatches, compared with other configurations.

D. Power Schedule

Power schedule is not mentioned in this article, which is commonly seen in fuzzing tools [23], [39]. If we treat fuzzing as an optimization problem, there are many methods to reach the desired optimum, including seed selection and mutation, coverage metrics, power schedule, etc. The key mechanism of this article is the VACO algorithm, which shortens the input length, increases coverage, and keeps the input sequence efficient in fuzzing the hardware. As the evaluation shows, VACO accomplishes these tasks with its internal design. Besides, there are other approaches in INSTILLER that make it more realistic in hardware fuzzing, including hardware-based seed selection and mutation. The above components contribute to the “efficient and realistic” fuzzing of INSTILLER. However, we are not claiming that the power schedule is not important in fuzzing. It is not the research focus of INSTILLER, so it is not included in this article. Besides, we are preparing to utilize the power schedule in hardware fuzzing and leave this as future work.

E. Remaining Challenges and Future Improvements

Remaining Challenges: The simulation of RTL execution is relatively slow compared with binary fuzzing, e.g., AFL. Binary fuzzing can reach the speed of thousands of executions per second, and RTL fuzzing can only reach one execution per several seconds. It is still a challenge to speed up the execution speed of RTL simulation.

In software instrumentation, LLVM and Clang greatly reduce the workload of practitioners. However, the instrumentation of CPU RTL requires FIRRTL. It is a time-consuming process to instrument the code.

Future Improvements: In the future, multiple ISAs can be added to INSTILLER, such as ARM and x86. It would be more attractive for manufacturers to invest in fuzzing these commercial ISAs. However, unlike RISC-V, it is relatively difficult for researchers to get access to these resources to conduct research.

We also plan to propose a new coverage mechanism, aiming to solve different types of bugs in fuzzing the CPU. For example, for bugs related to side channels, such as Meltdown, the coverage mechanism should focus on the branch predictions in the RTL.

VII. RELATED WORK

A. RTL Verification and Testing

RTL testing is a popular field in research, even in capture-the-flag competitions [40]. RFuzz utilizes mux coverage [9] to

fuzz CPU. DiFuzzRTL applies differential fuzz testing [10], which is a pioneer in this field. TheHuzz uses different coverage metrics [25] and conducts experiments to show their performance. SpecDoctor is an automated RTL fuzzer to find transient execution vulnerabilities [41]. GenFuzz is a GPU-accelerated hardware fuzzer with a genetic algorithm and multiple inputs [42]. Cascade uses asymmetric ISA presimulation to construct RISC-V programs [43]. INSTILLER is different from them, and we aim to shorten input instruction length and increase coverage.

HyperFuzzing converts hardware into software [44], which is a different approach to fuzz RTL. HFP also translates hardware to software to fuzz CPU [37]. Our tool does not need to translate hardware to software. These two tools are different from INSTILLER.

B. Coverage-Guided Grey-Box Fuzzing

CGF becomes prevalent since the release of AFL [23]. It discovers numerous bugs with its well-designed mechanism. The key to CGF is the coverage feedback to the fuzzer. AFLFast [27] is another milestone of CGF, which improves the power schedule (the number of executions on a seed). MOPT [32] automatically selects mutation operators using particle swarm optimization (PSO), which is a promising direction in fuzzing research. CollAFL proposes a coverage-sensitive fuzzing solution [45], which uses a finely designed coverage metric to effectively avoid path collisions in fuzzing. ovAFLow [46] deals with the problem of taint input bytes. It utilizes a lightweight fuzzing-based taint inference to guide the mutation strategy. INSTILLER is different from them, which utilizes the idea of CGF combined with differential testing to detect CPU bugs. More importantly, the goal of INSTILLER is to distill input instructions for more efficient hardware fuzzing, and it is different from the above fuzzers.

C. Optimization in Fuzzing

As a classical optimization technique, ACO is used in fuzzing. ACOFuzz uses ACO to allocate energy [47], which controls the power schedule of fuzzing. AFL-ant proposes a seed screening technique based on ACO [48]. This technique chooses the best seed with ACO. RGF concentrates on new code with an ACO-based power schedule [49]. Therefore, it is a directed fuzzer. In INSTILLER, based on the characteristics of RTL fuzzing, we propose a VACO to distill input instructions. Our VACO is different from the classic ACO in the above-related work. INSTILLER and these ACO-based fuzzers focus on different aspects of the fuzzing process.

Other optimization techniques can also be adopted in fuzzing. For example, EcoFuzz [28] uses a variant of multiarm bandit (MAB) to assign energy in fuzzing. The power schedule is altered according to the fuzzing status. MobFuzz [29] solves the problem of multiobjective optimization with a method called multiplayer MAB. The optimization technique in INSTILLER is ACO, which is different from the MAB algorithm, and we make modifications to it according to the situation of RTL fuzzing.

D. Seed Selection and Mutation

Seed selection and mutation strategies can improve fuzzing efficiency. TortoiseFuzz uses three metrics to select and prioritize seeds [50]. FairFuzz selects seeds that are chosen less frequently [33]. MemLock uses the amount of memory

consumption to detect memory consumption bugs [51]. FuzzFactory proposes waypoints to guide seed selection and improve fuzzing [52]. Waypoints can be memory consumption, algorithmic complexity, etc. MoonShine [53] optimizes OS fuzzer seed selection with trace distillation. It can distill seeds from system call traces of real-world programs while preserving the dependencies across the system calls. Herrera et al. [54] compared six seed selection approaches, concluding that fuzzing highly relies on the initial seed corpus. However, none of them is related to the heuristics of hardware, e.g., the number of jump instructions. INSTILLER uses hardware-related strategies, which improves the RTL fuzzing efficiency.

VIII. CONCLUSION

In this article, we conclude three challenges in RTL fuzzing in previous work, which include increasing input instruction length, no realistic interruption or exception, and no hardware-related seed selection and mutation. To solve these challenges, we propose input instruction relationship extraction, together with input instruction distillation based on a VACO. Moreover, we enable our fuzzer to include multiple interruptions and exceptions to cover more RTL states. We also propose hardware-related seed selection and mutation strategies to improve the fuzzing performance. In addition, we implement a prototype INSTILLER and conduct extensive experiments against state-of-the-art fuzzing work. The results show our tool outperforms previous work in coverage, input instruction length, and vulnerability discovery, which demonstrate the effectiveness of our proposed techniques.

ACKNOWLEDGMENT

Thanks to the reviewers' valuable comments and helpful suggestions.

REFERENCES

- [1] "Meltdown and Spectre." Accessed: Feb. 12, 2024. [Online]. Available: <https://meltdownattack.com/>
- [2] "The Pentium FDIV bug." Accessed: Feb. 12, 2024. [Online]. Available: <http://scihi.org/the-pentium-fdiv-bug/>
- [3] "Machine check exception on broadwell." Accessed: Feb. 12, 2024. [Online]. Available: <https://community.intel.com/t5/Processors/Machine-check-exception-on-Broadwell-quad-core-with-SpeedStep/mp/485622>
- [4] "Ryzen Segfault Bug." [Online]. Available: https://Ryzen_Processor_Segfault_Bug.html
- [5] "Intel Analysis of Speculative Execution Side Channels." Accessed: Feb. 12, 2024. [Online]. Available: <https://www.intel.cn/content/www/cn/zh/developer/articles/technical/software-security-guidance/technical-documentation/analysis-speculative-execution-side-channels.html>
- [6] D. Moundanos, J. A. Abraham, and Y. V. Hoskote, "Abstraction techniques for validation coverage analysis and test generation," *IEEE Trans. Comput.*, vol. 47, no. 1, pp. 2–14, Jan. 1998.
- [7] M. Kantrowitz and L. M. Noack, "I'm done simulating: now what? verification coverage analysis and correctness checking of the decchip 21164 alpha microprocessor," in *Proc. 33rd DAC*, 1996, pp. 325–330.
- [8] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of unix utilities," *Commun. ACM*, vol. 33, no. 12, pp. 32–44, 1990.
- [9] K. Laeuffer, J. Koenig, D. Kim, J. Bachrach, and K. Sen, "RFUZZ: Coverage-directed fuzz testing of RTL on FPGAS," in *Proc. ICCAD*, 2018, pp. 1–8.
- [10] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee, "DifuzzRTL: Differential fuzz testing to find cpu bugs," in *Proc. SP*, 2021, pp. 1286–1303.
- [11] C. Zhou, M. Wang, J. Liang, Z. Liu, and Y. Jiang, "Zeror: Speed up fuzzing with coverage-sensitive tracing and scheduling," in *Proc. 35th ASE*, 2020, pp. 858–870.
- [12] S. Nagy and M. Hicks, "Full-speed fuzzing: Reducing fuzzing overhead through coverage-guided tracing," in *Proc. SP*, 2019, pp. 787–802.
- [13] C. Zhang, W. Y. Dong, and Y. Zhu Ren, "INSTRCR: Lightweight instrumentation optimization based on coverage-guided fuzz testing," in *Proc. CCET*, 2019, pp. 74–78.

- [14] M. Wang et al., "RIFF: Reduced instruction footprint for Coverage-Guided fuzzing," in *Proc. USENIX ATC*, 2021, pp. 147–159.
- [15] M. Wang, J. Liang, C. Zhou, Z. Wu, X. Xu, and Y. Jiang, "Odin: On-demand instrumentation with on-the-fly recompilation," in *Proc. PLDI*, 2022, pp. 1010–1024.
- [16] S. Li and Z. Su, "Accelerating fuzzing through prefix-guided execution," in *Proc. PACMPL*, 2023, pp. 1–27.
- [17] H. Peng, Y. Shoshitaishvili, and M. Payer, "T-fuzz: Fuzzing by program transformation," in *Proc. SP*, 2018, pp. 697–710.
- [18] S. Schumilo, C. Aschermann, and R. Gawlik, "kAFL: Hardware-assisted feedback fuzzing for OS kernels," in *Proc. USENIX Security Symp.*, 2017, pp. 1–17.
- [19] C. Blum, "Ant colony optimization: Introduction and recent trends," *Phys. Life Rev.*, vol. 2, no. 2, pp. 353–373, 2005.
- [20] "RISC-V PLIC spec." Accessed: Feb. 12, 2024. [Online]. Available: https://riscv-plic-1.0.0_rc6.pdf
- [21] "The RISC-V instruction set manual volume II." Accessed: Feb. 12, 2024. [Online]. Available: <https://riscv-privileged-20190608-1.pdf>
- [22] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, User-Level ISA*, version, vol.-I, Univ. California, Berkeley, CA, USA, 2014.
- [23] "American fuzzy lop." Accessed: Feb. 12, 2024. [Online]. Available: <https://github.com/google/AFL>
- [24] W. M. McKeeman, "Differential testing for software," *Digit. Tech. J.*, vol. 10, no. 1, pp. 1–8, 1998.
- [25] R. Kande et al., "TheHuzz: Instruction fuzzing of processors using golden-reference models for finding software-exploitable vulnerabilities," in *Proc. USENIX Secur. Symp.*, 2022, pp. 1–18.
- [26] A. Lipowski and D. Lipowska, "Roulette-wheel selection via stochastic acceptance," *Physica A Statist. Mech. Appl.*, vol. 391, no. 6, pp. 2193–2196, 2012.
- [27] M. Böhme, V.-T. Pham, A. Roychoudhury, "Coverage-based greybox fuzzing as Markov chain," in *Proc. CCS*, 2016, pp. 1032–1043.
- [28] T. Yue et al., "EcoFuzz: Adaptive energy-saving greybox fuzzing as a variant of the adversarial multi-armed bandit," in *Proc. Usenix Secur. Symp.*, 2020, pp. 1–19.
- [29] G. Zhang et al., "Mobfuzz: Adaptive multi-objective optimization in gray-box fuzzing," in *Proc. NDSS*, 2022, pp. 24–28.
- [30] "Boom." Accessed: Feb. 12, 2024. [Online]. Available: <https://github.com/riscv-boom/riscv-boom>
- [31] "Rocket chip generator." Accessed: Feb. 12, 2024. [Online]. Available: <https://github.com/chipsalliance/rocket-chip>
- [32] C. Lyu et al., "MOPT: Optimized mutation scheduling for fuzzers," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1–19.
- [33] C. Lemieux and K. Sen, "Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage," in *Proc. ASE*, 2018, pp. 475–485.
- [34] "morlxx." Accessed: Feb. 12, 2024. [Online]. Available: <https://github.com/openrisc/morlxx>
- [35] "OpenRISC 1200 implementation." Accessed: Feb. 12, 2024. [Online]. Available: <https://github.com/openrisc/or1200>
- [36] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proc. CCS*, 2018, pp. 2123–2138.
- [37] T. Trippel and K. G. Shin, "Fuzzing hardware like software," in *Proc. USENIX Secur. Symp.*, 2022, pp. 1–19.
- [38] M. Dorigo, M. Birattari, and T. Stutzle, "Ant colony optimization," *IEEE Comput. Intell. Mag.*, vol. 1, no. 4, pp. 28–39, Nov. 2006.
- [39] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *Proc. WOOT*, 2020, pp. 1–12.
- [40] C. Chen, R. Kande, P. Mahmoodi, A.-R. Sadeghi, and J.V. Rajendran, "Trusting the trust anchor: Towards detecting cross-layer vulnerabilities with hardware fuzzing," in *Proc. 59th DAC*, 2022, pp. 1379–1383.
- [41] J. Hur, S. Song, S. Kim, and B. Lee, "Specdoctor: Differential fuzz testing to find transient execution vulnerabilities," in *Proc. CCS*, 2022, pp. 1473–1487.
- [42] D.-L. Lin, Y. Zhang, H. Ren, B. Khailany, S.-H. Wang, and T.-W. Huang, "GenFuzz: GPU-accelerated hardware fuzzing using genetic algorithm with multiple inputs," in *Proc. DAC*, 2023, pp. 1–6.
- [43] F. Solt, K. Ceesay-Seitz, and K. Razavi, "Cascade: CPU fuzzing via intricate program generation," in *Proc. USENIX Secur. Symp.*, 2024.
- [44] S. K. Muduli, G. Takhar, and P. Subramanyan, "Hyperfuzzing for SoC security validation," in *Proc. ICCAD*, 2020, pp. 1–9.
- [45] S. Gan et al., "CollAFL: Path sensitive fuzzing," in *Proc. SP*, 2018, pp. 679–696.
- [46] G. Zhang, P.-F. Wang, T. Yue, X.-D. Kong, X. Zhou, and K. Lu, "ovAFLow: Detecting memory corruption bugs with fuzzing-based taint inference," *J. Comput. Sci. Technol.*, vol. 37, pp. 405–422, Mar. 2022.
- [47] Y. Wu, Q. Zhan, H. Qu, and X. Zhao, "Acofuzz: Adaptive energy allocation for greybox fuzzing," in *Proc. ICSTW*, 2022, pp. 269–276.
- [48] B. Sun, B. Wang, B. Cui, and Y. Fu, "Greybox fuzzing based on ant colony algorithm," in *Proc. ICAINA*, 2020, pp. 1319–1329.
- [49] X. Zhu and M. Böhme, "Regression greybox fuzzing," in *Proc. CCS*, 2021, pp. 2169–2182.
- [50] Y. Wang et al., "Not all coverage measurements are equal: Fuzzing by coverage accounting for input prioritization," in *Proc. NDSS*, 2020, pp. 1–17.
- [51] C. Wen et al., "Memlock: Memory usage guided fuzzing," in *Proc. ICSE*, 2020, pp. 1–13.
- [52] R. Padhye, C. Lemieux, K. Sen, L. Simon, and H. Vijayakumar, "Fuzzfactory: Domain-specific fuzzing with waypoints," in *Proc. ACM Program. Lang.*, 2019, pp. 1–29.
- [53] S. Pailoor, A. Aday, and S. Jana, "MoonShine: Optimizing OS fuzzer seed selection with trace distillation," in *Proc. USENIX Secur. Symp.*, 2018, pp. 1–16.
- [54] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, and A. L. Hosking, "Seed selection for successful fuzzing," in *Proc. ISSTA*, 2021, pp. 1–14.
- [55] T.-Q. Zhou, Z.-P. Cai, F. Liu, and J.-S. Su, "In pursuit of beauty: Aesthetic-aware and context-adaptive photo selection in crowdsensing," *IEEE Trans. Knowl. Data Eng.*, vol. 35, pp. 9364–9377, 2023.



Gen Zhang received the Ph.D. degree in computer science and technology from the National University of Defense Technology, Changsha, China, in 2022.

His research interests include fuzzing and testing.



Pengfei Wang received the B.S., M.S., and Ph.D. degrees in computer science and technology from the National University of Defense Technology, Changsha, China, in 2011, 2013, and 2018, respectively.

His research interests include operating system and software testing.



Tai Yue received the B.S. degree in computer science and technology from Nanjing University, Nanjing, China, in 2017, and the M.S. degree in computer science and technology from the National University of Defense Technology, Changsha, China, in 2019.

His research interests include fuzzing and testing.



Danjun Liu received the Ph.D. degree in computer science and technology from the National University of Defense Technology (NUDT), Changsha, China, in 2023.

He is currently an Assistant Professor with NUDT. His research interests include operating systems and parallel computing.



Yubei Guo received the master's degree in computer science and technology from Hunan University, Changsha, China, in 2020.

Her research interests include security.



Kai Lu received the B.S. and Ph.D. degrees in computer science and technology from the National University of Defense Technology (NUDT), Changsha, China, in 1995 and 1999, respectively.

He is currently a Professor with NUDT. His research interests include operating systems, parallel computing, and security.